

Self-Healing 5G Timing Security

A plain-English walkthrough of the whole project — every part, the data, and the results

Project: AI-Native Self-Healing O-RAN Network using a Digital Twin
Prepared for the team — no telecom background needed
Revised 17 September 2026 (supersedes the 16 August edition)

Item	Status
Current phase	Standards-based software testbed (ITU-T G.8275.1) and a 132-run validation. Rule frozen 17 Sep 2026, 04:49:33 UTC
Headline result	Attacks caught: 60 of 60 . Benign faults called correctly: 48 of 60 (0.800). Honest "can't tell": 12 of 12 . Real hardware attack captures flagged: 6 of 6
Earlier phase (August)	Machine-learning pipeline. Commit 4e4bd16, tag known-good-2026-08-07. Its real-data percentages are no longer used as evidence (see §5)
Hardware (Tier-3)	Not started — the next phase
Note	Most build work was AI-generated under human direction. Every number here was re-checked against the raw files

1 · The project in 30 seconds

A 5G base station is split into pieces that talk over a link called the **fronthaul**. Those pieces must agree on **time** extremely closely. The ITU-T standards allow about **1.5 millionths of a second** of time error at the end of the network (G.8271.1), and they grade individual clocks in tens of **nanoseconds** (G.8273.2: 50, 20 or 10 ns). In published tests, a timing attack on the fronthaul dropped the cell about **2 seconds** after it started (TIMESAFE, ACM 2025).

The clock is shared using a protocol called **PTP** (IEEE 1588), which has no built-in security in normal use. So a faulty or malicious clock can knock a base station offline. The hard part: a harmless fault (a planned clock switch-over, network congestion) and a deliberate attack can look **almost identical**.

What the project is building: software that watches the timing, decides whether a problem is a **harmless fault** or a **real attack**, and then heals the network safely. If it can't tell, it says **"UNKNOWN"** instead of guessing. **What has been validated so far** is the *decision* part: fault or attack, on a standards-correct software testbed and on real hardware captures. The healing loop and digital twin were built in August, but they are not part of the September validation.

Where this sits in the bigger picture

- **Field:** 5G / O-RAN network security.
- **Zoom in:** the Open Fronthaul link between the radio and the baseband.
- **Zoom in again:** the **S-plane** — the traffic that carries the clock. That is our topic.

What is new about it

Telling faults from attacks is not a new idea in general, and GPS spoof-versus-outage in 5G was published in 2026. What is still open is doing it for the **PTP messages on the O-RAN fronthaul**. The main prior work, TIMESAFE, says that legitimate master changes and topology changes can resemble attacks, but its "benign" data is only healthy traffic. It never tests faulty-but-harmless events. That untested claim is the gap this project works on.

2 · The core problem: fault vs attack

When timing goes wrong, the symptom can look the same whatever the cause. The correct response, though, is **opposite** — so telling them apart is the whole game.

Harmless fault (benign)	Malicious attack
B2 — planned switch to the backup grandmaster during maintenance	A1 — a rogue device announces itself as the best clock
B3 — network congestion adds delay and jitter	A5 — an attacker floods the link with PTP messages
B7 — a link is re-wired during maintenance	A8 — a rogue boundary clock is inserted into the path
Planned boundary-clock replacement	A3 — old messages are recorded and replayed
GPS signal lost, clock rides its backup (<i>needs hardware</i>)	A2 — forged Sync messages carry a fake time

For a fault, the right move is usually to **wait it out**, because the real clock returns. For an attack, the right move is to **isolate the bad source and switch away**. Do the wrong one and you make the outage worse.

The headline experiment is A1 vs B2. In both, a new "best clock" appears. The packets alone cannot say which is which. The difference is **context the operator already has**: is that clock on the approved list (the *allow-list*), and is a maintenance window open?

3 · PTP in plain words

PTP is a conversation between a **master** (which has the right time) and **clients** (which want it). There are four main message types:

Message	What it says	How often (G.8275.1)
Announce	"I am a clock, and this is how good I am" (my quality and priority)	8 per second (one every 125 ms)
Sync + Follow_Up	"The time was T1 when I sent this"	16 per second
Delay_Req / Delay_Resp	The client asks "how long does the cable take?" and the master answers	16 per second

The client notes when Sync arrived (T2), sends Delay_Req (T3), and learns when the master got it (T4). The trip there and back is measured, and **half** of it is taken as the one-way delay, because the standard assumes both directions take the same time. **That assumption is exactly what a delay attack abuses.**

Who becomes the master? The election (BMCA)

Every clock sends Announce messages, and the clients run an election called the **BMCA** (Best Master Clock Algorithm). They compare fields such as **clockClass** (how good the time source is) and **priority2** (an operator's preference). The best one wins. A rogue device wins by *lying* in these fields.

The telecom rulebook we follow: ITU-T G.8275.1

- **domain 24** — the network "channel" number (the telecom range is 24–43).
- **priority1 is fixed at 128** — so any other value is a red flag.
- **clockClass 0–5 is reserved** (IEEE 1588 Table 5) — a real clock never sends it.
- Messages go to one fixed Ethernet group address, **01:1B:19:00:00:00**.
- Every relay (a **boundary clock**) adds 1 to **stepsRemoved**, so hop count is visible.
- Each sender numbers its messages with a **sequenceId** that only goes up. A replay makes it go backwards.

4 · How it works

Phase 1 (August): a machine-learning pipeline

Timing data was cut into 0.4-second windows, 28 numbers were computed per window, and a Random Forest labelled each window as fault or attack. There was also an "UNKNOWN" safety net, a digital twin that forecasts each fix, and a healing loop. Later checks showed that 12 of the 28 numbers only carry information in *simulated* data (GPS and oscillator fields are constant or missing in real captures). That is one reason Phase 2 was built.

Phase 2 (September): a standards-correct testbed and a frozen rule

The testbed. Six small virtual machines (Linux network namespaces) run the real **linuxptp** software, configured to G.8275.1:

```
[GM-A p2=100] [GM-B p2=110] -- bridge UP -- [Boundary Clock p2=120] -- bridge DOWN --  
[RU1] [RU2] [RU3]
```

- **GM-A** is the main grandmaster. **GM-B** is the approved backup, which makes planned failover (B2) possible.
- The **boundary clock** sits between two network segments, like O-RAN setups LLS-C2/C3, so hop counts change.
- **Three radio units** (RU1–RU3) are the clients.

The rule. Instead of a trained model, a fixed rule checks the evidence in this order. The first match wins:

- 1 Is any source sending Announce far faster than it says it will? → **A5 flood**
- 2 Does any sequenceId go backwards? → **A3 replay**
- 3 Is an unapproved clock relaying the real grandmaster with an extra hop? → **A8 rogue boundary clock**
- 4 Is an unapproved clock announcing *itself* as grandmaster? Is clockClass below 6, or priority1 not 128? → **A1 rogue master** (stepsRemoved ≥ 255 and alternateMasterFlag are also illegal and are flagged)
- 5 Is an unapproved source sending timing but never Announce? → **A2 Sync spoof**
- 6 Wrong domain, wrong group address, or wrong message rate? → **out-of-profile attack**
- 7 Grandmaster changed to an approved clock **during** maintenance? → **BENIGN (B2)**. Changed with **no** maintenance window? → **UNKNOWN**, because the packets can't tell a crash from an attack.
- 8 Otherwise, with only approved clocks and no violations → **BENIGN**.

Why a rule and not a model? Every constant in it comes from a standard (domain, priority1, clockClass, stepsRemoved, the 2× message-rate cap) or from operator configuration (the allow-list and maintenance window). There is **one** self-chosen value: a clock seen in under **1%** of Announce messages counts as a start-up blip. The standard's own rule (**2 Announce within 4 intervals**, IEEE 1588 §9.3.2.4) has been identified but not yet adopted, because changing the rule means re-freezing and re-running everything.

5 · The data — where the numbers come from

This is the question people ask most, so it's worth being exact. There are three sources, and they carry very different weight.

Source 1 — the August simulator (synthetic): not used as evidence

This is software that fakes a PTP clock with perfectly labelled faults. It was useful for building and debugging. A check found that one simulator field alone gives away the source: offsetScaledLogVariance is **4096** in every simulated row and **65535** in every real row. So any mix of simulated and real data can be told apart by that one number.

Source 2 — real TIMESAFE hardware captures (the real-world evidence)

These are actual PTP recordings of attacks on a real 5G testbed at Northeastern University, published with the TIMESAFE paper. Counted directly from the packets: the six attack captures hold **344,630** PTP packets, all on domain 24, with no optional add-on fields (no TLVs) at all.

Capture	PTP packets	Attack type
2024-10-16-announce_attack	260,525	Forged Announce (rogue master)
15min_announce_attack	46,998	Announce attack, fully standard-looking fields
prod_successful_announce_attack_ptp	13,565	Announce attack
2024-10-08-sync_attack_singlestep_1	9,078	Sync attack (one-step)
2024-10-08-sync_attack_1	8,440	Sync attack (two-step)
2024-10-08-announce_attack_1	6,024	Announce attack

What the attacker actually sends (read from the raw fields of the 2024-10-16 capture): source b8cef6ffff5e6afa, one character different from b8cef6fffe5e6afa, claims to be grandmaster with **priority1 = 0**, **priority2 = 0**, **clockClass = 0**, in 24,653 Announce messages. clockClass 0 is illegal. The approved grandmaster fcfa6afffe02babe advertises 128 / 120 / 6.

A duplicate-file trap we found. The folders held 21 capture files, but only **14 are unique**. Three "separate" sessions (announce_session_1, announce_session_2 and a UE-data capture) are byte-for-byte the same file as 15min_announce_attack. The August edition counted "five independent sessions", so some of its "unseen session" tests may have been run on data the model had already seen.

Source 3 — our own G.8275.1 testbed (the September evidence)

11 scenarios × 12 repeats = 132 runs. Each repeat uses fresh random attacker identities, priorities, timings and burst sizes (run/randparams.py), so the 12 repeats are genuinely different. The August netem data could not be reused. It announced once every **2 s** instead of every **125 ms**, a 16× mismatch with the real captures, so any classifier could have told the two apart by rhythm alone.

6 · Results

How to read the numbers

- **Sensitivity** = of the real attacks, how many were caught.
- **Specificity** = of the harmless faults, how many were correctly left alone.
- **95% range** (Wilson interval) = where the true value probably lies, given a limited number of runs. Narrower is more certain.
- The rule was **frozen** (fingerprinted with SHA-256, 18 files) **before** these runs and never changed after. Earlier runs that exposed bugs are development data and are not counted.

The 132-run campaign (repeats 30–41)

Measure	Result	95% range	Plain meaning
Sensitivity	1.000 (60/60)	0.940–1.000	Every attack run was flagged
Specificity, all benign	0.800 (48/60)	0.682–0.882	12 harmless runs were wrongly flagged
Specificity without the BC-swap case	1.000 (48/48)	0.926–1.000	All 12 mistakes come from one scenario
Honest "UNKNOWN"	1.000 (12/12)	0.757–1.000	Unplanned crash: never guessed
Naming the right attack	0.817 (49/60)	0.701–0.894	Detected, but sometimes given the wrong name

Per attack, correctly named: A1 8/12 · A2 12/12 · A3 12/12 · A5 12/12 · A8 5/12. When a rogue master or rogue boundary clock appears, the radio units switch to it, and that makes their message numbers genuinely restart. The replay check (step 2) fires first, so A1 and A8 are sometimes named "A3". The attack is still caught.

Why specificity is 0.800, not 1.000. In the *planned boundary-clock replacement* scenario, the replacement clock **is** approved: it is written into the context file as a second boundary clock. But the frozen rule reads only **one** approved boundary-clock identity, so the legitimate new one looks like a rogue one. This is a limit of the rule's input format, not a detection failure. It was **reported, not patched**: fixing it after seeing the result would turn test data into tuning data.

Real hardware captures — rule applied unchanged

Capture	Caught by the standards alone?	Verdict
2024-10-16-announce_attack	Yes — clockClass < 6 and priority1 ≠ 128 (24,653 each)	ATTACK
2024-10-08-announce_attack_1	Yes — clockClass < 6 and priority1 ≠ 128 (457 each)	ATTACK

Capture	Caught by the standards alone?	Verdict
prod_successful_announce_attack_ptp	Yes — clockClass < 6 (39)	ATTACK
2024-10-08-sync_attack_1	Yes — sequenceId goes backwards (2,500)	ATTACK
2024-10-08-sync_attack_singlestep_1	Yes — sequenceId goes backwards (2,455)	ATTACK
15min_announce_attack	No — every field looks legal; needs the allow-list	ATTACK

6 of 6 flagged; 5 of 6 by the standards alone. The sixth is a "healthy-looking spoof", exactly the case that needs operator context. **Rule-breaking test messages:** 7 of 7 caught, each by its own cited rule (illegal clockClass, alternateMasterFlag, stepsRemoved $\geq$ 255, wrong domain, wrong group address, priority1 $\neq$ 128, and one clean control).

Earlier (August) results — kept for the record, no longer used as evidence

August reported 99.96% (Announce), 99.66% (Sync/Follow_Up) and 99.66% (single-step) protection on real data, false alarms cut from 4.49% to 2.37%, simulator accuracy 0.991, recovery in 0.933 s, and live decisions in 0.081 s on average (0.404 s at worst) over 5,665 windows. Those numbers are **not re-used**, for three reasons: some "unseen" sessions were duplicates of training files; 12 of the 28 features carry information only in simulated data; and the benign netem data announced at the wrong rate. The September campaign does **not** re-measure decision time. It judges each run as a whole.

7 · Bugs we found — and fixed

August: the "silent zero" safety flaw

Under heavy packet loss, the timing tool returned nothing, the software filled in a zero, and a zero looks exactly like perfect timing. So the system reported 'healthy' at the very moment it had gone blind. **Fix:** treat missing data as its own state, and never call anything healthy without proof. This is called **failing closed**. The September rule keeps this principle: missing evidence gives **UNKNOWN**, never BENIGN.

September: found while building the testbed (all fixed before the freeze)

What went wrong	Why it mattered	Fix
The replay check tracked Delay_Resp by the <i>answering</i> clock	The standard echoes the <i>asking</i> client's number, so a healthy run showed 799 false "replays"	Track by the asking client
The allow-list only checked grandmaster names	A spoofer sending 1,800 fake Sync messages and no Announce was invisible	Check every sender
A8 was first built as a one-port fake master	That is just A1 again: the same attack built twice	Rebuilt as a real two-port relay
The G.8275.1 election setting was missing	linuxptp ran the ordinary election, not the telecom one. A capture can't show this	Added dataset_comparison G.8275.x
PATH_TRACE was switched on	Real captures never carry it, so it gave away "testbed vs real"	Switched off
Software sometimes started before the virtual link	Empty captures	Wait for the link; one retry; failures discarded openly

Four "give-away" fields (**provenance leaks**) were found in total: the 16× rhythm mismatch, the 4096/65535 simulator constant, PATH_TRACE, and the PTP minor-version number (testbed 1, real 0). The first three are gone. The last one is built into the software, so it is excluded from any mixed dataset.

8 · What's done, what isn't, and what's next

Area	Status
G.8275.1 software testbed	Built and validated: 132 runs, zero infrastructure failures, rule frozen before the runs
Real-capture check	6 of 6 flagged by the unchanged rule
Rule-breaking message tests	7 of 7
Test catalogue	168+ cases researched: 17 run, 106 specified but not run, 45 need hardware
Hardware (Tier-3)	Not started — timing NIC, second machine, GPS grandmaster

Is the software testbed "complete with no compromise"? Not quite, and it is safer to say so. What is true: everything that was built follows the standard, was run properly, and reports its limits. **Still open in software:** (1) a list of approved boundary clocks, which would fix all 12 false alarms; (2) adopting the standard's foreign-master rule and the other identified checks ($\pm 30\%$ timing, full clockClass list, VLAN/unicast); (3) better attack naming (0.817); (4) using the logged pmc management data; (5) the O-RAN tests for packet removal and malformed messages. Items 1 and 2 need a new freeze and a full re-run.

What software can never do here

- All virtual machines share **one crystal**, so real clock drift (B6) can't happen. A **second PC** fixes this.
- There is no hardware timestamping. Measured noise: offset median **0.9 μ s** (95th percentile 5.6 μ s), cable-delay estimate median **7.2 μ s**, against a nanosecond target. A timing NIC (for example Intel i210/i226) fixes this and makes delay attacks measurable.
- GPS spoofing, jamming and holdover, and SyncE, need real GPS and physical-layer hardware. A second PC alone does not unlock them.

One-sentence summary: on a standards-correct software testbed and on real 5G hardware captures, a frozen, standards-based rule caught every attack and correctly refused to guess when the packets couldn't decide. It wrongly flagged one harmless scenario (a planned boundary-clock swap) for a known, named reason. Hardware testing and the open software items above are the next steps.

9 · Mini-glossary

Term	Meaning
O-RAN	Open, multi-vendor way of building 5G base stations.
Fronthaul	The link between the radio unit and the baseband unit.
S-plane	The fronthaul traffic that carries the clock. Our topic.
PTP (IEEE 1588)	The protocol that shares time across the network.
G.8275.1	The ITU-T telecom rulebook for PTP: domain 24–43, 8 Announce/s, 16 Sync/s, priority1 = 128.
Grandmaster (GM)	The top clock everyone follows.
Boundary clock (BC)	A relay clock between network segments. It adds 1 to stepsRemoved.
Announce / Sync	"I am a clock, this good" / "the time was T1".
BMCA	The 'best clock wins' election that attackers exploit by lying.
clockClass / priority1 / priority2	Fields used in the election. clockClass 0–5 is illegal; G.8275.1 fixes priority1 at 128.
sequenceId	A message counter that only goes up. Going backwards suggests a replay.
Allow-list	The operator's list of approved clocks.
Maintenance window	A declared period when planned changes are expected.
GNSS / GPS, SyncE	Satellite time source / physical-layer frequency sharing. Both need hardware.
Holdover	Running on an internal backup clock after losing the reference. Benign.
Sensitivity / specificity	Share of attacks caught / share of harmless events correctly left alone.
Wilson 95% range	The likely range of the true value, given the number of runs.
Freeze (pre-registration)	Fingerprinting the rule before testing, so it can't be tuned to the answers.
Provenance leak	A field that reveals <i>where</i> data came from rather than what happened.
Fail-closed / UNKNOWN	Refusing to say 'healthy' (or guess) when the evidence isn't there.
TIMESAFE	The 2025 ACM study whose real hardware attack captures we test against.
Digital twin	A fast model that predicts what a fix will do before it is applied (August phase).